A Formal Verification of TCP (RFC 793) in Lean 4

Samuel Schlesinger

March 26, 2026

Abstract

We present a machine-checked formalization of the Transmission Control Protocol as specified in RFC 793. Using the Lean 4 theorem prover, we model the TCP state machine, define segment acceptability predicates over modular 32-bit sequence number arithmetic, and prove correctness properties of connection establishment (three-way handshake), graceful teardown, and global safety invariants including an unconditional system invariant, send window bounds, and sequence number monotonicity. We prove that reaching ESTABLISHED state requires segment delivery from SYN-SENT or SYN-RECEIVED—the states produced by the three-way handshake. The formalization uses a two-endpoint system model with an explicit network, capturing reordering, loss, and duplication. All theorems are verified by Lean's kernel with zero uses of `sorry`—if the code typechecks, the proofs hold.

Contents

1 Introduction **2**

2 Sequence Numbers **2**

3 Core Types **3**

4 State Machine **3**
4.1 Endpoint and System Model . . . 3
4.2 Event Processing . . . 4
4.3 SystemStep . . . 4

5 Segment Acceptability **4**

6 Connection Establishment **5**

7 Connection Teardown **5**

8 Invariants and Safety Properties **5**

9 Conclusion **6**
9.1 Design Decisions . . . 6
9.2 Limitations and Known Deviations . . . 7
9.3 Future Work . . . 7

1 Introduction

The Transmission Control Protocol (TCP) is the foundation of reliable communication on the Internet. Originally specified by Postel [2], TCP provides ordered, reliable, byte-stream delivery over an unreliable network. Despite its ubiquity, TCP's specification is given in informal prose, and subtle corner cases—simultaneous open, old duplicate SYN recovery, half-open connection detection, FIN-with-data—have historically been sources of implementation bugs.

This work presents a machine-checked formalization of TCP in Lean 4. Our contributions are:

1. A **modular sequence number arithmetic** library with a circular less-than relation and proofs of irreflexivity, decidability, and arithmetic interaction properties.
2. **Core type definitions** faithfully modeling RFC 793 segments, the Transmission Control Block (TCB), and the eleven connection states.
3. A **two-endpoint system model** with an explicit network (segment multiset) and a `SystemStep` inductive type capturing delivery, user calls, timeouts, loss, and duplication.
4. The **full event-processing function** implementing RFC 793 §3.9: all user calls (OPEN, SEND, RECEIVE, CLOSE, ABORT, STATUS), segment arrival in CLOSED, LISTEN, SYN-SENT, and the 8-check pipeline for remaining states, and all three timeout types.
5. **Segment and ACK acceptability** predicates with the four-case table from RFC 793 §3.3 and lemmas connecting them to TCB invariants.
6. **Handshake correctness**: the three-way handshake reaches ESTABLISHED with synchronized sequence numbers, and RST correctly handles stale connections.
7. **Teardown correctness**: graceful close transitions through the correct states, FIN occupies one sequence number, and TIME-WAIT behaves correctly.
8. **Safety invariants**: no data before ESTABLISHED, `SND.UNA` $\leq$ `SND.NXT` preservation across handshake and close, ACK acceptability preservation, an unconditional system invariant with send window bounds, and `SND.UNA`/`RCV.NXT` monotonicity.
9. **Handshake safety property**: ESTABLISHED is only reachable via segment delivery from SYN-SENT or SYN-RECEIVED. No user call or timeout enters ESTABLISHED. Separate lemmas show SYN-SENT requires an active OPEN and SYN-RECEIVED requires receiving a SYN.

All proofs compile with zero uses of `sorry`. The formalization targets Lean 4 v4.28.0 without Mathlib.

2 Sequence Numbers

TCP sequence numbers are 32-bit unsigned integers with modular arithmetic (RFC 793 §3.3). The key challenge is defining a meaningful ordering: since the space wraps around, the standard total order on natural numbers is inappropriate.

Definition 2.1 (SeqNum)**.** A `SeqNum` is a wrapper over `UInt32`:

```
structure SeqNum where
  val : UInt32
```

Definition 2.2 (Circular Less-Than)**.** $a < b$ iff b is strictly ahead of a within the forward half-space:

```
def lt (a b : SeqNum) : Bool :=
  let d := b.val - a.val
  d != 0 && d < halfSpace  -- halfSpace = 2^31
```

This corresponds to $(b - a) \bmod 2^{32} \in (0, 2^{31})$.

Theorem 2.1 (Irreflexivity)**.** `SeqNum.lt` is irreflexive: $\forall a,\ \neg(a < a)$. This follows because $a - a = 0$, and the predicate requires $d \neq 0$. Proved as `SeqNum.lt_irrefl`.

Theorem 2.2 (Decidability)**.** `SeqNum.lt` is decidable, with a `Decidable` instance provided for use in `if` expressions and `decide` tactics.

We also prove interaction lemmas: `add_zero`, `diff_self`, and `le_refl`.

3 Core Types

The `Tcp.Types` module defines the fundamental structures.

Definition 3.1 (Segment)**.** A TCP segment models the header fields from RFC 793 §3.1: source/destination ports, sequence number, acknowledgment number, control bits (URG, ACK, PSH, RST, SYN, FIN), window size, urgent pointer, and payload data. Wire-format fields (checksum, data offset, options) are omitted as they concern encoding, not protocol logic.

Definition 3.2 (SEG.LEN)**.** Segment length includes data octets plus SYN and FIN contributions to sequence space (RFC 793 §3.3): $\texttt{SEG.LEN} = |\texttt{data}| + \texttt{SYN} + \texttt{FIN}$.

Definition 3.3 (TCB)**.** The Transmission Control Block stores all send and receive variables from RFC 793 §3.2: `SND.UNA`, `SND.NXT`, `SND.WND`, `SND.UP`, `SND.WL1`, `SND.WL2`, `ISS`, `RCV.NXT`, `RCV.WND`, `RCV.UP`, and `IRS`.

Definition 3.4 (TcpState)**.** The eleven TCP states: Closed, Listen, SynSent, SynReceived, Established, FinWait1, FinWait2, CloseWait, Closing, LastAck, TimeWait.

4 State Machine

4.1 Endpoint and System Model

A `TcpEndpoint` bundles the connection state, TCB, open mode (active/passive), send queue, receive buffer, reassembly buffer, retransmission queue, and FIN tracking fields.

The `System` structure models two communicating endpoints with an explicit network:

```
structure System where
  endpointA : TcpEndpoint
  endpointB : TcpEndpoint
  network : List Segment
```

The network is treated as an unordered multiset: any segment may be delivered next. This is the standard approach in protocol verification.

4.2 Event Processing

Event processing follows RFC 793 §3.9 faithfully:

- **User calls**: OPEN (active/passive), SEND (segmentize and transmit), RECEIVE (deliver buffered data), CLOSE (send FIN), ABORT (send RST), STATUS.
- **Segment arrival in CLOSED**: discard data; respond with RST unless the incoming segment is RST.
- **Segment arrival in LISTEN**: ignore RST; reject ACK with RST; process SYN by sending SYN-ACK and entering SYN-RECEIVED.
- **Segment arrival in SYN-SENT**: the 5-step pipeline (ACK check, RST check, security, SYN check, drop).
- **8-check pipeline** for remaining states: (1) sequence check, (2) RST, (3) security (skipped), (4) SYN, (5) ACK + window update, (6) URG, (7) segment text, (8) FIN.
- **Timeouts**: user timeout (abort), retransmission (resend head of queue), TIME-WAIT (close).

4.3 SystemStep

The `SystemStep` inductive relates system states via atomic transitions: segment delivery to either endpoint, user calls, timeouts, segment loss, and segment duplication. A `SystemTrace` is the reflexive-transitive closure.

5 Segment Acceptability

Definition 5.1 (Segment Acceptability)**.** The four-case table from RFC 793 §3.3:

SEG.LEN	RCV.WND	Test
0	0	$\texttt{SEG.SEQ} = \texttt{RCV.NXT}$
0	> 0	$\texttt{RCV.NXT} \leq \texttt{SEG.SEQ} < \texttt{RCV.NXT} + \texttt{RCV.WND}$
> 0	0	not acceptable
> 0	> 0	start or end of segment in window

Definition 5.2 (ACK Acceptability)**.** Two variants are defined:

- **ESTABLISHED+**: $\texttt{SND.UNA} < \texttt{SEG.ACK} \leq \texttt{SND.NXT}$ (strict left inequality—duplicate ACKs excluded).
- **SYN-RECEIVED**: $\texttt{SND.UNA} \leq \texttt{SEG.ACK} \leq \texttt{SND.NXT}$ (non-strict—allows $\texttt{SEG.ACK} = \texttt{ISS}$).

Lemma 5.1 (Zero-length, zero-window)**.** A zero-length segment with zero window is acceptable iff $\texttt{SEG.SEQ} = \texttt{RCV.NXT}$. Proved as `segmentAcceptable_zero_zero`.

Lemma 5.2 (Data with zero window)**.** A data-bearing segment is never acceptable when $\texttt{RCV.WND} = 0$. Proved as `segmentAcceptable_data_zeroWnd`.

Lemma 5.3 (ACK preserves invariant)**.** If an ACK is acceptable (ESTABLISHED-style), advancing `SND.UNA` to `SEG.ACK` preserves $\texttt{SND.UNA} \leq \texttt{SND.NXT}$. Proved as `ackAcceptable_preserves_invariant`.

6 Connection Establishment

Theorem 6.1 (Three-Way Handshake Correctness)**.** Starting from A = CLOSED and B = LISTEN, after A performs active OPEN, the SYN is delivered to B, the SYN-ACK is delivered to A, and the ACK is delivered to B: both endpoints reach ESTABLISHED with A.`tcb.irs` = B.`tcb.iss` and B.`tcb.irs` = A.`tcb.iss`. Proved symbolically as `threeWayHandshake_correct` for arbitrary ISS values, using a two-pass simplification strategy.

Theorem 6.2 (SYN Occupies Sequence Space)**.** After sending a SYN, `SND.NXT` = `ISS` + 1. Proved symbolically as `syn_occupies_sequence_space`.

Theorem 6.3 (Simultaneous Open)**.** When both endpoints perform active OPEN and receive the other's SYN in SYN-SENT, each transitions to SYN-RECEIVED (not ESTABLISHED), as expected for the first phase of simultaneous open. Proved as `simultaneousOpen_enters_synReceived`.

Theorem 6.4 (ESTABLISHED Requires Handshake)**.** An endpoint can only reach ESTABLISHED by processing a delivered segment while in SYN-SENT or SYN-RECEIVED state. No user call, timeout, or other state transition enters ESTABLISHED. Entering SYN-SENT requires an active OPEN; entering SYN-RECEIVED requires receiving a SYN (proved as `processSegmentListen_syn`); and without SYN, the SYN-SENT endpoint cannot enter ESTABLISHED (proved as `processSegmentSynSent_not_est`
Together these lemmas show that the three-way handshake is the only path to ESTABLISHED. Proved as `established_requires_handshake_step`, `established_only_from_pre_established`, `processUserCall_not_established`, and `processTimeout_not_established`.

Theorem 6.5 (RST Handling)**.** A valid RST in SYN-RECEIVED with passive origin returns the endpoint to LISTEN. A valid RST in ESTABLISHED aborts the connection. Proved as `rst_synReceived_passive_returns_to_listen` and `rst_established_aborts`.

7 Connection Teardown

Theorem 7.1 (CLOSE from ESTABLISHED)**.** CLOSE from ESTABLISHED transitions to FIN-WAIT-1, emits a FIN segment, and advances `SND.NXT` by 1 (FIN occupies one sequence number). Proved as `close_established_enters_finWait1` and `fin_occupies_sequence_space`.

Theorem 7.2 (CLOSE-WAIT → LAST-ACK)**.** CLOSE from CLOSE-WAIT enters LAST-ACK (not CLOSING). The RFC 793 §3.9 text says "enter CLOSING state" but Figure 6 (the state diagram) shows CLOSE-WAIT → LAST-ACK. We follow Figure 6 as authoritative. Proved as `close_closeWait_enters_lastAck`.

Theorem 7.3 (TIME-WAIT Behavior)**.** TIME-WAIT timeout transitions to CLOSED. RST in TIME-WAIT causes immediate transition to CLOSED. Proved as `timeWait_timeout_closes` and `rst_timeWait_closes`.

8 Invariants and Safety Properties

Theorem 8.1 (No Data Before ESTABLISHED)**.** In CLOSED, LISTEN, SYN-SENT, and SYN-RECEIVED states, the RECEIVE call never returns data to the user. Data piggybacked on SYN segments is buffered in the reassembly buffer and not delivered until the connection reaches ESTABLISHED. Proved as `no_data_in_closed`, `no_data_in_listen`, `no_data_in_synSent`, `no_data_in_synReceived`.

Theorem 8.2 (Sequence Number Monotonicity)**.** `SND.UNA` $\leq$ `SND.NXT` is preserved by every processing function. Per-function preservation theorems are proved symbolically: `processOpen_preserves`, `processSegment_preserves`, `processClose_preserves'` (using the window invariant to eliminate the external send-window hypothesis), `processSend_preserves`, `processTimeout_preserves`, etc.

Theorem 8.3 (ACK Acceptability Preserves Monotonicity)**.** If an ACK is acceptable (ESTABLISHED-style), advancing `SND.UNA` to `SEG.ACK` maintains `SND.UNA` $\leq$ `SND.NXT`. This is the key lemma for proving monotonicity inductively over system traces. Proved as `ackAdvanceTcb_preserves`.

Theorem 8.4 (Window Invariant)**.** In-flight bytes (`SND.NXT` $-$ `SND.UNA`) remain bounded within the forward half-space with room for future advances. Pre-FIN states maintain a bound of $+2 < 2^{31}$; post-FIN states (where no more data will be sent) maintain $+1 < 2^{31}$. This is proved preserved by every processing function. Proved as `windowInvariant` with per-function preservation theorems.

Theorem 8.5 (SystemStep Invariant (Unconditional))**.** The combined system invariant—`SND.UNA` $\leq$ `SND.NXT`, the window bound, and the ISS structural invariant for SYN-SENT—is preserved by every `SystemStep` transition *unconditionally*, with no blanket hypotheses. The window invariant provides the half-space room needed for CLOSE's FIN, eliminating the need for external send-window assumptions. Proved as `systemStep_preserves_invariant`.

Theorem 8.6 (SND.UNA Monotonicity)**.** For any `SystemStep`, each endpoint's `SND.UNA` is monotonically non-decreasing (modulo connection teardown to CLOSED or fresh connection initialization from CLOSED/LISTEN). Proved as `systemStep_sndUna_monotone`.

Theorem 8.7 (RCV.NXT Monotonicity)**.** For any `SystemStep` with well-formed segments (data length $< 2^{31}-1$), each endpoint's `RCV.NXT` is monotonically non-decreasing (modulo connection teardown or initialization from CLOSED/LISTEN/SYN-SENT). Proved as `systemStep_rcvNxt_monotone`.

Theorem 8.8 (Send Window Invariant under Cooperative Peer)**.** Under the cooperative peer assumption (RFC 1122 [1] §4.2.2.16), in the data-transfer states (ESTABLISHED and CLOSE-WAIT), `SND.NXT` $\leq$ `SND.UNA` $+$ `SND.WND`. The cooperative predicate (`windowNonShrinking`) requires that the peer's advertised right edge (`SEG.ACK` $+$ `SEG.WND`) covers all sent data (`SND.NXT`). This is preserved by every `SystemStep`: user calls and timeouts unconditionally, segment delivery under the cooperative assumption. The proof also requires the combined `endpointInvariant` to hold on the processing endpoint. Proved as `systemStep_preserves_sendWindow` with zero `sorry`.

9 Conclusion

We have presented a machine-checked formalization of TCP (RFC 793) in Lean 4, covering modular sequence number arithmetic, the full event-processing state machine, segment acceptability, connection establishment, graceful teardown, and safety invariants including a cooperative send window bound (RFC 1122 [1] §4.2.2.16). All proofs compile with zero `sorry`.

9.1 Design Decisions

Key modeling choices include: representing sequence numbers as `UInt32` wrappers with a circular half-space comparison; using a two-endpoint system model with an explicit segment network; implementing event processing as pure functions with nondeterminism confined to the `SystemStep` inductive; and omitting security/precedence checks (RFC 793 §3.6) as orthogonal IP-layer concerns.

9.2 Limitations and Known Deviations

This formalization intentionally excludes several RFC 793 features: security/compartment/precedence (§3.6), checksum computation, MSS option negotiation, ISN clock-based generation (§3.3), RTO calculation (§3.7), and IP-layer interactions.

Known deviations from RFC 793:

- **CLOSE-WAIT + CLOSE**: the §3.9 text says "enter CLOSING state" but Figure 6 (the state diagram) shows CLOSE-WAIT → LAST-ACK. We follow Figure 6 as authoritative. This is a known errata in RFC 793.
- **Urgent pointer interpretation**: RFC 793 §3.9 says `RCV.UP <- max(RCV.UP, SEG.UP)`, where `SEG.UP` is defined as a positive offset from `SEG.SEQ`. We interpret this as `RCV.UP <- max(RCV.UP, RCV.NXT + SEG.UP)`, converting the offset to a sequence number. This is consistent with the idealized segment assumption (`SEG.SEQ = RCV.NXT`) and is a known RFC ambiguity.
- **No timer state**: the formalization does not model the retransmission timer, 2×MSL timer, or user timeout timer as explicit state. Timeouts are modeled as nondeterministic events via `SystemStep`.
- **No segment reassembly**: the model assumes in-order segment delivery for data processing. Out-of-order segments are buffered in `reassemblyBuf` but never reassembled; the `pipelineText` function appends data directly to `recvBuffer` and advances `RCV.NXT` by the segment's data length. A full reassembly implementation would require tracking sequence number ranges and merging overlapping segments, which is orthogonal to the protocol state machine properties proved here.

9.3 Future Work

Natural extensions include: congestion control (RFC 5681); window scaling and timestamps (RFC 7323); modeling of the full simultaneous open handshake completion through SYN-RECEIVED to ESTABLISHED; segment reassembly for out-of-order delivery; and data delivery correctness (proving that the received byte stream matches the sent byte stream in order).

References

[1] Robert Braden. Requirements for Internet Hosts — Communication Layers. RFC 1122, Internet Engineering Task Force, October 1989. STD 3.

[2] Jon Postel. Transmission Control Protocol. RFC 793, Information Sciences Institute, University of Southern California, September 1981. DARPA Internet Program Protocol Specification.